

PHASE 11 — CBOM DATA MODEL, SCHEMA & ASSET NORMALIZATION

Project: Enterprise Cryptographic Discovery & Analysis Tool (ECDAT)

Problem Statement: SIH26164

Organization: National Technical Research Organisation (NTRO)

Theme: Blockchain & Cybersecurity

Phase: 11 of 22

Document Type: Technical Research & Engineering Handbook

Status: Canonical Data Architecture

1. Purpose of This Phase

Phase 10 established how ECDAT discovers cryptographic artifacts across an enterprise.

We now have scanners producing observations from:

```
Source Code
Binaries
Libraries
Containers
Certificates
Network
Cloud
Kubernetes
HSM/KMS
SBOM
Runtime
```

But there is a fundamental engineering problem.

Different scanners may describe the same thing differently.

For example:

```
Scanner A:
RSA-2048

Scanner B:
```

RSA / 2048

Scanner C:
rsaEncryption

Scanner D:
RSA2048

Scanner E:
OID 1.2.840.113549.1.1.1

ECDAT must understand that these may all represent:

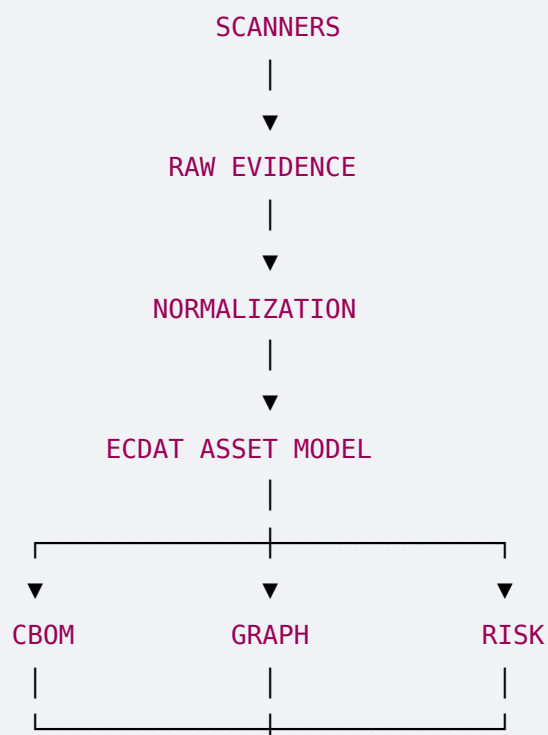
Algorithm:
RSA
Key Size:
2048

Therefore we need one canonical model.

The ECDAT Cryptographic Asset Model.

2. Why the Data Model Matters

The data model becomes the backbone connecting every subsystem.





If this model is poorly designed, every later phase becomes difficult.

3. Core Design Principle

ECDAT should separate:

What was observed

from:

What the system believes it represents

and from:

What risk that representation creates

Therefore:

Observation



Asset



Assessment



Recommendation

These should not be collapsed into one record.

4. Four-Layer Data Architecture

A clean model contains four layers.

Layer 1 — Evidence

What did a scanner observe?

Layer 2 — Asset

What real-world entity does that evidence represent?

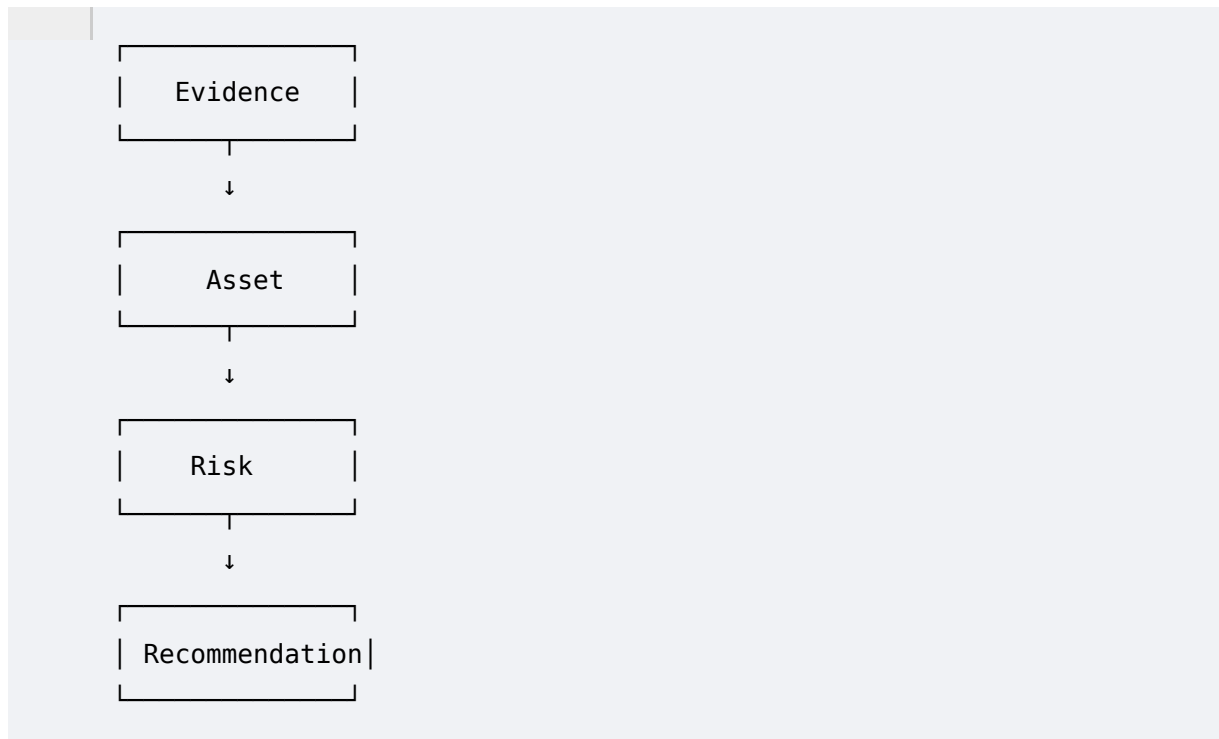
Layer 3 — Assessment

What does ECDAT conclude about it?

Layer 4 — Decision

What should the organization do?

Conceptually:



5. What Is a Cryptographic Asset?

A cryptographic asset is an identifiable entity involved in cryptographic protection or cryptographic infrastructure.

Examples:

```
Algorithm
Key
Certificate
Protocol
Cipher Suite
Crypto Library
Crypto API
HSM
KMS Key
Random Number Generator
Signature
Hash
Key Exchange
```

Not everything detected by a scanner should automatically become a cryptographic asset.

6. Asset Categories

ECDAT should classify assets into major categories:

```
ALGORITHM
KEY
CERTIFICATE
PROTOCOL
CIPHER_SUITE
LIBRARY
CRYPTO_API
CRYPTO_IMPLEMENTATION
HSM
KMS_KEY
RNG
APPLICATION
CONTAINER
BINARY
DATA_ASSET
```

7. Asset Identity

Every asset needs a stable identifier.

Example:

```
asset_id:
crypto-algorithm-8f4d92...
```

The ID should not depend solely on display names.

For example:

```
"RSA"
```

is not enough to uniquely identify:

```
RSA-2048
RSA-3072
RSA-4096
```

8. Canonical Identity

A canonical identity can be generated from normalized attributes.

Example:

```
Algorithm
+
Family
+
Parameters
+
Standard
```

Conceptually:

```
canonical_id =
hash(
    type +
    normalized_name +
    parameters
)
```

This allows duplicate detection.

9. Evidence Object

Every discovery event should produce an evidence object.

Example:

```
{
  "evidence_id": "ev-00123",
  "type": "source_code",
  "source": "static-analyzer",
  "file": "src/security/signing.py",
  "line_start": 42,
  "line_end": 42,
  "observed": "ECDSA.sign()",
  "confidence": 0.98
}
```

Evidence should remain immutable.

10. Evidence vs Asset

Suppose three scanners discover RSA.

You might have:

```
Evidence 1 → RSA
Evidence 2 → RSA
Evidence 3 → RSA
```

These should resolve into:

```
          RSA Asset
        /   |   \
       /    |    \
Evidence1 Evidence2 Evidence3
```

Thus:

Multiple observations can support one asset.

11. Assessment Object

After collecting evidence, ECDAT assesses the asset.

Example:

```
{
  "asset_id": "crypto-123",
  "algorithm": "ECDSA",
  "status": "confirmed",
  "confidence": 0.97,
  "quantum_status": "vulnerable"
}
```

Assessment may change over time.

12. Risk Object

Risk should be separate.

Example:

```
{
  "risk_id": "risk-789",
  "asset_id": "crypto-123",
  "severity": "high",
}
```

```
"quantum_risk": "high",  
"business_impact": "critical"  
}
```

This avoids contaminating the asset model with constantly changing risk calculations.

13. Recommendation Object

Example:

```
{  
  "recommendation_id": "rec-111",  
  "source_algorithm": "ECDSA",  
  "target": "ML-DSA",  
  "strategy": "hybrid",  
  "confidence": 0.91  
}
```

14. Migration Object

Migration represents execution.

Example:

```
{  
  "migration_id": "mig-221",  
  "source": "ECDSA",  
  "target": "ML-DSA",  
  "status": "planned",  
  "owner": "PKI Team"  
}
```

15. Canonical ECDAT Asset

A canonical asset could conceptually look like:

```
{  
  "id": "asset-123",  
  "type": "algorithm",  
  "name": "ECDSA",  
  "family": "digital-signature",  
  "parameters": {  
    "curve": "P-256"  
  }  
}
```



```
    },  
    "standard": "FIPS 186",  
    "status": "active",  
    "quantum_status": "vulnerable",  
    "evidence": [],  
    "confidence": 0.98  
  }  
}
```

16. Algorithm Schema

Algorithm assets should include:

```
id  
name  
family  
function  
parameters  
key_size  
security_level  
standard  
implementation  
status  
quantum_status
```

Example:

```
Algorithm:  
AES  
  
Function:  
Encryption  
  
Mode:  
GCM  
  
Key Size:  
256  
  
Quantum Status:  
Not broken by Shor's algorithm
```

17. Algorithm Families

ECDAT should classify algorithms into families.

Symmetric

AES
ChaCha20

Hash

SHA-256
SHA-384
SHA-512

Digital Signature

RSA-PSS
ECDSA
EdDSA
ML-DSA
SLH-DSA

Key Establishment

ECDH
DH
ML-KEM

This enables risk grouping.

18. Parameters

Algorithm identity must include relevant parameters.

Examples:

RSA:
2048 / 3072 / 4096

ECDSA:
P-256 / P-384 / P-521

AES:
128 / 192 / 256
GCM / CBC / CTR

Two configurations of the same algorithm can have different security properties.

19. Key Schema

Keys should contain metadata such as:

```
key_id
algorithm
key_type
size
curve
purpose
usage
location
owner
creation_time
expiration_time
rotation_policy
status
```

Example:

```
{
  "key_id": "key-981",
  "algorithm": "RSA",
  "size": 2048,
  "purpose": "digital-signature",
  "location": "HSM-03"
}
```

20. Private Key Protection

ECDAT should never need to store private key material for normal inventory.

Instead:

```
Private Key:
NOT EXTRACTED
```

Metadata:
Collected

This is a critical security boundary.

21. Certificate Schema

A certificate object could contain:

```
certificate_id
fingerprint
subject
issuer
serial_number
valid_from
valid_until
public_key_algorithm
public_key_size
signature_algorithm
key_usage
extended_key_usage
san
environment
```

22. Certificate Example

```
{
  "certificate_id": "cert-123",
  "fingerprint": "sha256:...",
  "subject": "api.example.internal",
  "issuer": "Enterprise-CA",
  "public_key_algorithm": "ECDSA",
  "curve": "P-256",
  "signature_algorithm": "SHA256withECDSA",
  "valid_until": "2027-06-01"
}
```

23. Certificate vs Public Key Algorithm

These are different properties.

Example:

Public Key:

ECDSA P-256

Certificate Signature:

RSA-PSS

ECDAT must preserve both.

Otherwise the inventory becomes misleading.

24. Protocol Schema

Protocol objects can contain:

```
protocol
version
endpoint
cipher_suites
key_exchange
authentication
certificate
configuration
```

Example:

TLS 1.3

Cipher:

TLS_AES_256_GCM_SHA384

Key Exchange:

ECDHE

Authentication:

ECDSA

25. Cipher Suite Schema

A cipher suite may contain:

```
name
protocol
encryption
authentication
```

```
key_exchange
hash
quantum_status
```

Example:

```
TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384
```

should be decomposed into its cryptographic components.

26. Library Schema

Library objects should include:

```
library_id
name
version
ecosystem
package
vendor
license
source
crypto_capabilities
```

Example:

```
{
  "name": "OpenSSL",
  "version": "3.x",
  "ecosystem": "native",
  "crypto_capabilities": [
    "RSA",
    "ECDSA",
    "AES",
    "SHA-2"
  ]
}
```

27. Library vs Algorithm

Important distinction:

```
OpenSSL
```

is not an algorithm.

It is an implementation/library containing many algorithms.

Therefore:

```
Application
  ↓
DEPENDS_ON
  ↓
OpenSSL
  ↓
IMPLEMENTS
  ↓
ECDSA
```

28. Crypto API Schema

An API-level asset could contain:

```
api_id
name
library
function
language
operation
algorithm
```

Example:

```
EVP_DigestSignInit()
```

could map to:

```
Operation:
Digital Signature

Library:
OpenSSL
```

29. Custom Crypto Implementation

A custom implementation should have:

```
implementation_id
location
language
operation
suspected_algorithm
confidence
review_status
```

Example:

```
Potential Custom Crypto

Algorithm:
Unknown

Confidence:
0.62

Review:
Required
```

30. Application Schema

Application objects should include:

```
application_id
name
version
environment
business_criticality
owner
business_unit
repository
deployment
```

31. Container Schema

Container assets should contain:


```
image_id
repository
tag
digest
base_image
os
architecture
packages
libraries
application
```

Example:

```
sha256:abc123...
```

should be preferred over a mutable tag as the immutable identity.

32. Binary Schema

Binary objects can contain:

```
binary_id
filename
hash
format
architecture
compiler
libraries
imports
embedded_certificates
crypto_findings
```

33. Data Asset Schema

A data asset could contain:

```
data_id
name
classification
sensitivity
retention
location
```

```
owner
business_process
```

Example:

```
Customer Financial Records
```

```
Classification:
Highly Sensitive
```

34. Relationship Schema

Relationships should be first-class objects.

Example:

```
Application
  —USES—>
ECDSA
```

Metadata:

```
relationship_id
source_id
target_id
type
confidence
evidence
first_seen
last_seen
```

35. Why Relationships Need IDs

Suppose:

```
Application A USES RSA
```

Today.

Tomorrow:

```
Application A no longer USES RSA
```

The historical relationship should remain available.

Therefore relationship identity and temporal validity matter.

36. Temporal Model

Every asset or relationship can contain:

```
first_seen
last_seen
observed_at
valid_from
valid_until
```

Example:

```
RSA Usage

First Seen:
2026-01-10

Last Seen:
2026-08-24

Status:
Active
```

37. Provenance Model

Every important field should ideally have provenance.

Example:

```
algorithm = ECDSA

Source:
Certificate Parser

Evidence:
certificate fingerprint X
```

Another:

```
algorithm = ECDSA
```

```
Source:
AI inference
```

```
Evidence:
signing.py:42
```

These should not be treated identically.

38. Confidence Model

Confidence can be calculated from evidence.

For example:

```
Certificate parser:
0.99
```

```
Static API:
0.95
```

```
Dependency:
0.90
```

```
AI:
0.80
```

Combined evidence could produce:

```
Final Confidence:
0.98
```

The exact mathematical model should be documented and calibrated.

39. Evidence Weighting

Possible hierarchy:

```
Direct Cryptographic API
↓
Certificate / Protocol Evidence
↓
Binary Symbol Evidence
↓
```

```
Dependency Evidence
  ↓
Configuration Evidence
  ↓
AI Semantic Inference
  ↓
Naming / Heuristic Evidence
```

This is not an absolute ranking, but a useful design principle.

40. Normalization

Normalization converts heterogeneous input into canonical values.

Example:

```
Input:
SHA256

Input:
SHA-256

Input:
sha_256

Input:
OID representation
```

becomes:

```
SHA-256
```

41. Algorithm Alias Registry

ECDAT should maintain an alias registry.

Example:

```
RSA-SHA256
sha256WithRSAEncryption
1.2.840.113549.1.1.11
```

These can map to:

```
RSA + SHA-256
```

This registry becomes extremely valuable.

42. OID Mapping

Certificates and cryptographic protocols often use OIDs.

ECDAT should maintain mappings such as:

```
OID
  ↓
Algorithm
  ↓
Parameters
```

This helps certificate analysis.

43. Library Name Normalization

Examples:

```
openssl
OpenSSL
libssl
libcrypto
```

must be carefully distinguished.

For example:

```
OpenSSL
```

may refer to a project, while:

```
libcrypto
```

is a specific component.

ECDAT should preserve both canonical identity and display names.

44. Version Normalization

Versions can appear as:

```
3.0.12
v3.0.12
OpenSSL_3_0_12
```

These should normalize to:

```
3.0.12
```

while preserving original evidence.

45. Environment

Every asset should know its environment where possible:

```
development
testing
staging
production
disaster-recovery
unknown
```

The same algorithm in development and production can have radically different risk.

46. Criticality

Assets should support business criticality:

```
low
medium
high
critical
unknown
```

This should ideally come from organizational metadata rather than AI guessing.

47. Data Classification

Possible classifications:

```
Public
Internal
Confidential
Sensitive
```

Highly Sensitive
Restricted

Again, organizational policy should define the actual taxonomy.

48. Quantum Status

The schema needs an explicit quantum assessment.

Possible values:

```
quantum_safe  
quantum_vulnerable  
partially_affected  
context_dependent  
unknown
```

This should not be confused with classical security.

49. Classical Security Status

An algorithm can be:

```
Classically Secure  
Quantum Vulnerable
```

For example, traditional public-key algorithms may remain classically secure today while being vulnerable to sufficiently capable quantum computers.

ECDAT needs both dimensions.

50. Lifecycle Status

Assets can have:

```
active  
deprecated  
legacy  
retired  
unknown
```

This is essential for migration planning.

51. Risk Schema

Risk should include:

```
risk_id
asset_id
risk_type
severity
likelihood
impact
quantum_risk
business_impact
confidence
calculated_at
```

Potential risk types:

```
Quantum
Weak Algorithm
Short Key
Expired Certificate
Unsupported Library
Custom Crypto
Misconfiguration
Migration Risk
```

52. Mosca Assessment Schema

Since the problem statement explicitly mentions Mosca's algorithm, the data model should represent its inputs.

Conceptually:

```
data_lifetime
+
migration_time
+
estimated_qc_arrival
```

ECDAT can store:

```
{
  "data_lifetime_years": 15,
```

```
"migration_time_years": 5,  
"estimated_qc_horizon_years": 10  
}
```

Then classify the risk.

The exact methodology should be transparent rather than presenting the result as a precise prediction.

53. Recommendation Schema

Recommendations can include:

```
recommendation_id  
asset_id  
current_algorithm  
target_algorithm  
migration_strategy  
rationale  
compatibility  
performance  
cost  
confidence
```

54. Migration Strategy

Possible strategies:

```
replace  
upgrade  
hybrid  
parallel  
reconfigure  
wrap  
decommission  
monitor
```

55. Migration Constraints

Recommendations should capture constraints:

```
latency
hardware
protocol
library
compliance
interoperability
vendor
performance
key-management
```

Example:

```
ML-DSA recommended
```

```
Constraint:
Legacy client interoperability
```

Therefore:

```
Strategy:
Hybrid
```

56. CBOM Structure

The overall CBOM could look like:

```
CBOM
|
├─ Metadata
├─ Applications
├─ Algorithms
├─ Keys
├─ Certificates
├─ Libraries
├─ Protocols
├─ Containers
├─ Binaries
├─ Evidence
├─ Relationships
├─ Risks
```

57. Interoperability

ECDAT should avoid inventing an isolated format unnecessarily.

The design should consider existing SBOM/CBOM ecosystems, especially:

```
CycloneDX
SPDX
```

CycloneDX is particularly relevant because it has evolved toward software and cryptographic component modeling.

ECDAT can extend an established format rather than creating an incompatible format from scratch.

58. ECDAT Extensions

ECDAT-specific fields may include:

```
quantum_status
mosca_assessment
migration_priority
pqc_recommendation
business_criticality
cryptographic_centrality
migration_blockers
```

These can be represented as extensions where appropriate.

59. JSON Representation

A simplified ECDAT record:

```
{
  "asset": {
    "id": "crypto-001",
    "type": "algorithm",
    "name": "ECDSA",
    "parameters": {
      "curve": "P-256"
    }
  }
}
```

```

    },
    "assessment": {
      "quantum_status": "quantum_vulnerable",
      "confidence": 0.98
    },
    "risk": {
      "severity": "high",
      "business_impact": "critical"
    }
  },
  {
    "quantum_status": "quantum_vulnerable",
    "confidence": 0.98,
    "severity": "high",
    "business_impact": "critical"
  }
]

```

60. Database Model

A relational implementation might contain:

```

assets
evidence
relationships
applications
algorithms
keys
certificates
libraries
protocols
containers
risks
recommendations
migrations
scans

```

Foreign keys connect the entities.

61. Graph Model

The same information can become:

```

graph TD
    Application((Application))
    ECDSA((ECDSA))
    OpenSSL((OpenSSL))
    CustomerData((CustomerData))

    Application -- USES --> ECDSA
    Application -- DEPENDS_ON --> OpenSSL
    Application -- PROTECTS --> CustomerData

```

```
(ECDSA)
|
└─MIGRATES_TO─>(ML-DSA)
```

Thus:

The relational and graph representations should describe the same canonical domain model.

62. API Objects

API responses should use stable schemas.

Example:

```
GET /api/assets/{asset_id}
```

Response:

```
{
  "id": "crypto-001",
  "type": "algorithm",
  "name": "ECDSA",
  "quantum_status": "quantum_vulnerable",
  "confidence": 0.98,
  "evidence_count": 14,
  "application_count": 37
}
```

63. Scan Object

A scan should have:

```
scan_id
scanner
target
started_at
completed_at
status
files_scanned
findings
errors
coverage
```

64. Finding Object

Finding:

```
{
  "finding_id": "finding-001",
  "type": "crypto_usage",
  "asset_id": "crypto-001",
  "location": "src/signing.py:42",
  "confidence": 0.98,
  "status": "confirmed"
}
```

65. Schema Evolution

The schema will evolve.

Version it:

```
ECDAT Schema v1
ECDAT Schema v2
ECDAT Schema v3
```

Never silently change semantics.

66. Backward Compatibility

Old scan results should remain readable.

Example:

```
v1 scan
  ↓
Compatibility Layer
  ↓
Current ECDAT Model
```

This is important for long-lived enterprise deployments.

67. Data Validation

Every incoming object should be validated.

Examples:

```
Algorithm type valid?  
Version valid?  
Required fields present?  
Relationship targets exist?  
Confidence within 0-1?  
Dates valid?
```

Invalid data should not enter the canonical graph.

68. Import / Export

ECDAT should support exporting:

```
JSON  
CSV  
CBOM  
PDF Report  
Graph Data
```

Import could support:

```
SBOM  
CBOM  
Certificate inventory  
Cloud inventory  
Existing asset lists
```

69. Data Quality Pipeline

Complete flow:

```
Raw Evidence  
  ↓  
Parsing  
  ↓  
Normalization  
  ↓  
Validation  
  ↓  
Entity Resolution  
  ↓  
Deduplication
```


↓
Canonical Asset
↓
Graph / CBOM

70. Duplicate Resolution Example

Suppose:

```
Repository Scanner:  
OpenSSL 3.0.8  
  
Container Scanner:  
openssl-3.0.8  
  
Binary Scanner:  
libcrypto.so.3
```

ECDAT combines evidence.

Potential final representation:

```
Library:  
OpenSSL  
  
Version:  
3.0.8  
  
Components:  
libcrypto  
libssl
```

The exact mapping should depend on evidence.

71. Unknown Values

A major principle:

Unknown is better than invented.

If ECDAT cannot determine:

```
Algorithm:
```

Unknown

rather than:

Algorithm:
RSA

because a heuristic guessed it.

72. Partial Knowledge

ECDAT should support partial information.

Example:

Operation:
Digital Signature

Algorithm:
Unknown

Library:
OpenSSL

Confidence:
0.71

This is a legitimate result.

Later evidence can improve it.

73. Evidence Accumulation

Suppose initial scan:

Algorithm:
Unknown

Confidence:
0.65

Later:

Certificate:
ECDSA P-256

Then:

```
Algorithm:  
ECDSA  
Confidence:  
0.97
```

The asset should evolve while retaining history.

74. Contradictory Evidence

Suppose:

```
Static analysis:  
RSA  
  
Runtime:  
ECDSA
```

ECDAT should not hide this.

Instead:

```
Conflict:  
Detected  
  
Possible Explanation:  
Runtime configuration selects ECDSA.
```

AI can investigate the conflict.

75. Confidence and Contradictions

Example:

```
RSA  
Static:  
0.92  
  
Runtime:  
0.98 ECDSA
```

Final assessment:

Runtime-selected algorithm:

ECDSA

Static possible algorithm:

RSA

This is much more accurate than choosing one silently.

76. Data Lineage

ECDAT should be able to answer:

Why does the dashboard say this application uses ECDSA?

The lineage:

Dashboard

↓

Risk

↓

Asset

↓

Finding

↓

Evidence

↓

Source File

↓

Line 42

This is essential for security audits.

77. Auditability

Every important change should be logged:

Who

What

When

Why

Source

Previous Value

New Value

Example:

```
Risk changed:
Medium → High

Reason:
New quantum assessment model
```

78. Data Retention

ECDAT should define retention policies for:

```
Raw Scan Artifacts
Evidence
Assets
Historical Relationships
Risk Assessments
Reports
Audit Logs
```

Retention requirements may vary by deployment.

79. Privacy

The model should avoid storing unnecessary sensitive data.

For example:

Do:

```
Private Key:
Exists
Location:
HSM-02
```

Do not:

```
Private Key:
<actual key>
```

unless a specific authorized workflow requires something beyond inventory.

80. Security Labels

Assets can include:

```
classification:  
PUBLIC  
INTERNAL  
CONFIDENTIAL  
RESTRICTED
```

This allows policy-based access.

81. Canonical Asset Lifecycle

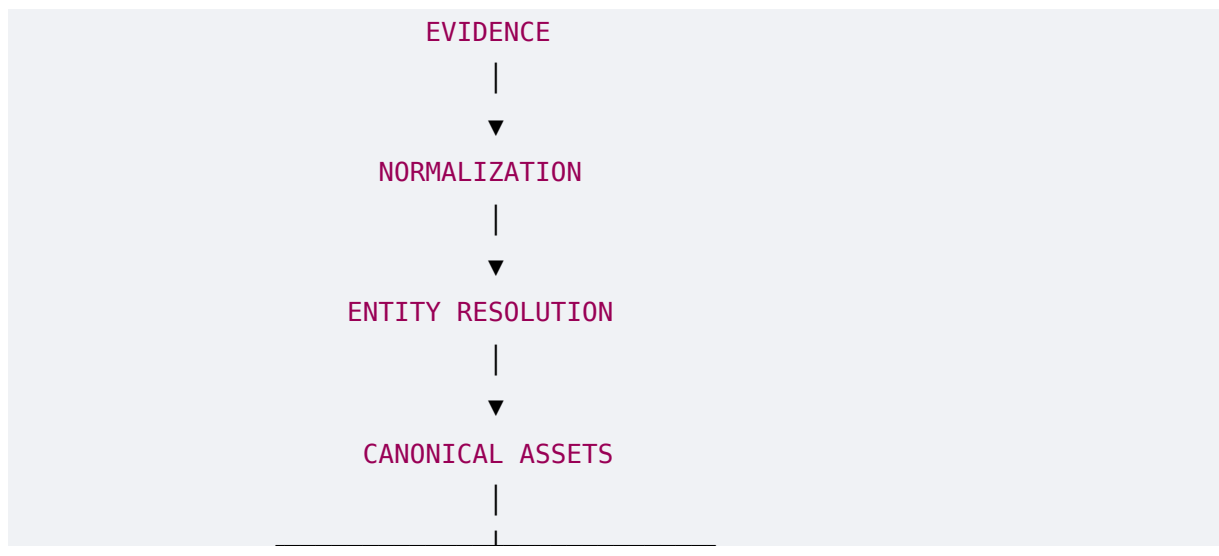
The complete lifecycle becomes:

```
DISCOVERED  
↓  
OBSERVED  
↓  
NORMALIZED  
↓  
IDENTIFIED  
↓  
CORRELATED  
↓  
ASSESSED  
↓  
RISK-RATED  
↓  
RECOMMENDED  
↓  
MIGRATING
```

This is an important foundation for later phases.

82. Complete Canonical Architecture

```
RAW SOURCES  
|  
▼  
SCANNERS  
|  
▼
```



83. Requirements Derived From Phase 11

ECDAT needs a canonical model supporting:

Core Assets

- Algorithms
- Keys
- Certificates
- Libraries
- Protocols
- Cipher suites
- APIs
- Applications
- Containers
- Binaries
- Data assets

Evidence

- Source location
- Scanner
- Timestamp
- Confidence
- Provenance
- Hash
- Raw observation

Intelligence

- Quantum status
- Classical security
- Risk
- Criticality
- Mosca inputs
- Recommendations

Relationships

- Uses
- Depends on
- Protects
- Issues
- Stores
- Runs on
- Calls
- Migrates to

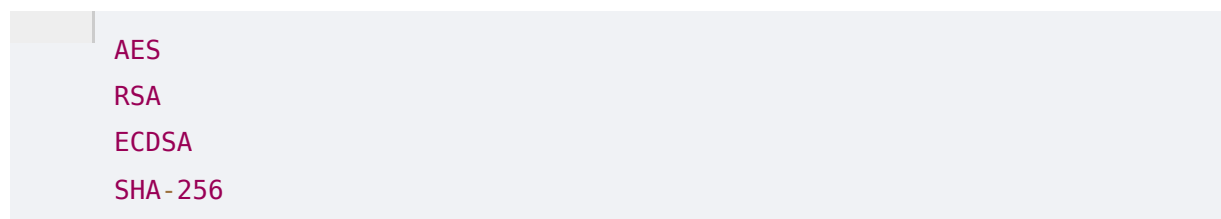
Governance

- Ownership
- Classification
- Versioning
- Audit
- Retention
- Access control

84. The Most Important Design Decision

ECDAT should **never make the CBOM merely a list of algorithms.**

A weak CBOM:



A useful CBOM:

Application A

- └─ Uses ECDSA P-256
- └─ Uses AES-256-GCM
- └─ Uses SHA-256
- └─ Depends on OpenSSL 3.x

ECDSA

- └─ Protects Payment API
- └─ Certificate: cert-001
- └─ CA: CA-01
- └─ Quantum Status: Vulnerable
- └─ Recommendation: ML-DSA / Hybrid

The second is an actual enterprise cryptographic inventory.

85. The ECDAT Canonical Chain

At this stage, the entire project is becoming:



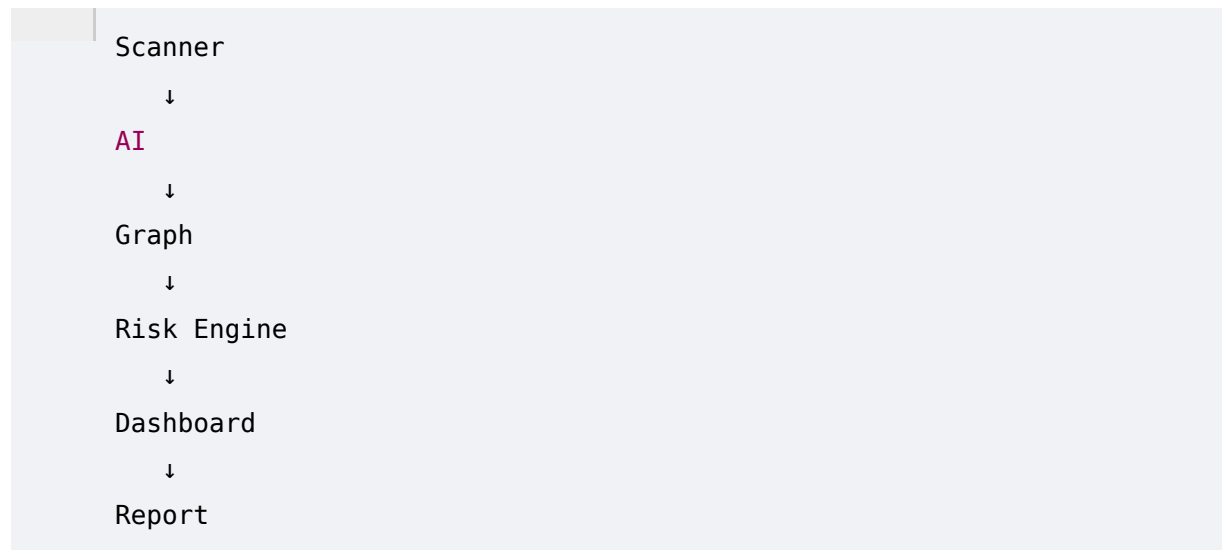
This is the core lifecycle of the platform.

86. Phase 11 Final Takeaway

The single most important principle from this phase is:

Every cryptographic conclusion must be representable as structured, versioned, evidence-backed data.

That allows:



to all operate on the **same source of truth**.

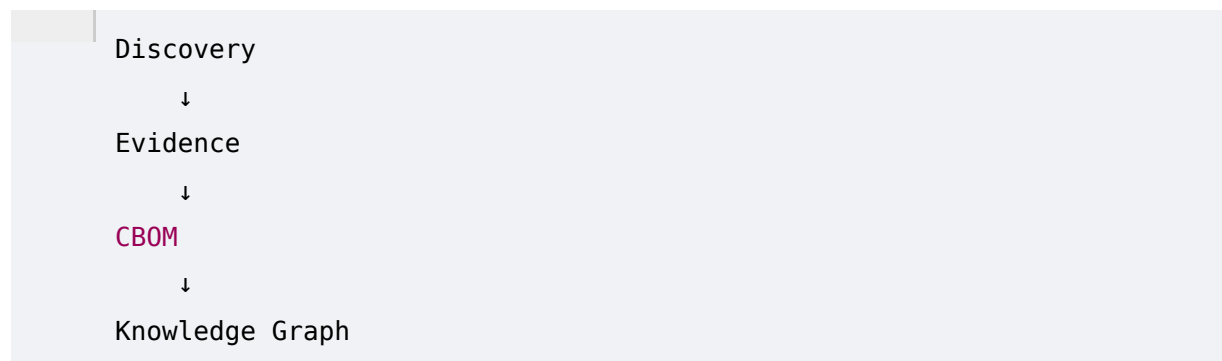
Without this canonical model, every subsystem would create its own interpretation of the same cryptographic asset.

With it:

ECDAT gets one enterprise-wide cryptographic truth layer.

PHASE 12 PREVIEW — QUANTUM RISK ENGINE, MOSCA MODEL & RISK SCORING

Now we have:



The next question is the heart of the NTRO problem statement:

Which cryptographic assets are actually at risk, how serious is that risk, and which ones must be migrated first?

Phase 12 will design the **Quantum Risk Engine**, including:

- Classical cryptographic risk
- Quantum cryptographic risk
- Shor's algorithm impact
- Grover's algorithm impact
- Harvest Now, Decrypt Later
- Data lifetime
- Cryptographic lifetime
- Migration time
- Mosca's theorem / inequality
- Quantum threat horizon
- Algorithm risk scoring
- Key-size risk
- Protocol risk
- Certificate risk
- Library risk
- Business criticality
- Data sensitivity
- Exposure
- Dependency centrality
- Migration complexity
- Risk propagation
- Risk aggregation
- Risk matrices
- Priority scoring
- Executive risk scores
- Technical risk scores
- Scenario analysis
- Confidence intervals
- What-if analysis
- Risk visualization

- Risk ranking
- Migration prioritization

The key output will be something like:

QUANTUM RISK ENGINE	
Asset:	ECDSA P-256
Quantum Risk:	CRITICAL
Data Sensitivity:	HIGH
Business Critical:	CRITICAL
Migration Time:	4 years
Data Lifetime:	12 years
Mosca Assessment:	ACTION REQUIRED
Priority:	P1

And more importantly, ECDAT will be able to answer:

"Out of everything we discovered, what should the organization migrate first, and why?"

That will turn the inventory into an actual **decision engine**.

Phase 11 complete.